Q1 Database Design-------

```
create table category(
cid int primary key,
cname varchar(50) not null,
ctype varchar(50) default "Electronics",
pid int,
foreign key (cid) references product(pid));

create table product(
pid int primary key,
pname varchar(50) not null,
price double(12,2) check price>0
);

create table supplier(
sid int primary key,
sname varchar(50) not null);

create table stockTransaction(
stid int primary key,
currentStock int,
pid int,
foreign key (sid) references product(pid));
```

Q2. SQL Query Writing -------

1) select max(salary) as second_highest_salary from Employees where salary < (select max(salary) from Employees);

2) select avg(salary) from Employees group by DepartmentId having avg(salary) > 50000;

3) select * from Employees where JoiningDate >= currDate() - interval 2 year;

4) select e.Name as employee, m.Name as manager from employee e left join employee m on e.EmployeeId = m.EmployeeId ;

Q3. Advanced SQL

1) create view Display as

Select p.pname, c.cname, st.currentStock from product p join category c on p.pid=c.pid, product p join stockTransaction st on p.pid=st.pid;

2)

```
delimiter//

create procedure insertion(

in vid int,

in vname varchar(50),
```

```
in vprice double(12,2));

begin //
insert into product (pid, pname, price) values (vid, vname, vprice);
end;
delimiter;
```

3)

```
delimiter//
create trigger updation(
id int, salary double(12,2));

begin //
if old.salary <> new.salary then
insert into auditTable(eid,old.salary) values (id,old.salary);
end if;
end;
delimiter;
```

4)

```
start transaction //
update salary = salary - 5000;
if salary < 5000 then
Rollback ();
```

```
end if;

Commit();

end;
```

Q4. SQL Concepts

1)Indexes help us to retrieve the data fastly without scanning the whole table. If a table has lakhs of records when we use select it scans the whole data from start to required data, but when we use indexes it will pinpoint the record, so that the time will be saved.

We should only use indexes when we want to search for specific record in large data. We should avoid them in small data because indexes are only useful in scanning, it will become difficult when we add new data or modify the data.

2)Where and Having both are used for condition data retrieval but where is used for single rows, it doesn't work on aggregated functions or groupby data. Whereas having clause is used for multiple columns like aggregated functions and groupby data. Among both the clauses where clause has the highest preference than Having clause.

3)Clustered indexes are unique, primary key is a clustered index. We can retrieve the data fastly and each record has its own index so that there is no space for duplicate date.

Non-Clustered index are composite keys, they can be mutated and can be repeated.

4)A transaction can be either fully successful or a failure. when we update the transaction after only it gets succeeded succefully we use commit, if it encounters any errors we use Rollback. When we use commit, the transaction is saved permanently internally we cannot undo the transaction. Before using commit if any transaction faces failure we can use Rollback and can move to the previous transaction.When we use Rollback, it gets rolled back to the previous transaction internally, which is succesfull.

SECTION B - ASP.NET Core & C#

Q5. C# Programming

```
create class Employee(
id int,
name string,
salaray double)
```

Q6.

```
using Microsoft.AspNetCore.Mvc;
namespace WebAPI.Controllers
{
[ApiController]
[Route("api/[Controller]")]
```

```
public class StudentController : ControllerBase
{
private static List<StudentController> students = new List<StudentController>()
{
new student{
Id=1,
Name="Adithya"
}
};
[HTTPGet]
public IActionResult GetAll()
{
return ok(students);
}
[HTTPGet("{id}")]
public IActionResult GetById(int id)
{
var student = students.FirstOrDefault(x=>x.Id==id);
if(student==null)
return NotFound("Student Not Found");
return ok(students)
}
}
```

Q7.

1) DbContext class manages the whole database. for example, if a library is a database and the shelves are dbset then the librarian is the DbContext, which manages all the records.

2)

3) Add-Migration syncs our data with the database, like if we change the data in our system it also changes the data in database, it keeps our data updated.

Update-Database is used to update the database when we make any changes to our data

4) controllers should not directly access DbContext because it can access all the data which is accessed by DbContext then there might be data breach

Q8)

A secured API uses JWT Authentication.

A client calls GET /api/students but receives 401 Unauthorized.

The main reason is the client doesn't have the access to the server. When we use JWT Authentication, it produces a key to access the server. If the client key, who is trying to access the data is different from the key which is produced by JWT, then it denies the access to the client and they may get 401 Unauthorized.

Another reason is client may entered his details like his username, or password or his access key wrong, then the server may restrict the client from accessinf=g